AI loop engineering: from zero to enterprise — Advanced

Portfolio projects · lessons 19–30

Each takes 1–3 days and ends with something to show an employer.

Project 1: Refund desk as a killable, crash-safe, traced fleet

Goal: Turn the Brightside refund desk into a three-worker Docker Compose fleet that survives worker kills with zero duplicate refunds, stops within one iteration on a kill switch, and reports cost per successful refund; the drill numbers in the README are what a staff engineer asks for first.

Uses lessons: 20, 22, 26a, 26b, 27, 28a, 28b, 29a, 30

1. Start from the chapter 3 desk loop and add a SQLite (or Postgres) runs table with columns run_id, tenant_id, owner, lease_expires_at, attempt and status; add a refunds table with a unique key on (tenant_id, order_id, idempotency_key).

2. Derive the refund idempotency key from run_id and order_id, then replay the same refund 200 times from two threads and confirm the refunds table holds exactly 1 row per order.

3. Serve POST /runs, GET /runs/{id} and POST /runs/{id}/approval from FastAPI; refunds above 100 USD pause at a LangGraph interrupt until the approval endpoint answers.

4. Set LEASE_TTL_S=2 and HEARTBEAT_S=0.5, then run docker compose up --scale worker=3 --wait and kill one worker at a random point in 50 drills; record takeover latency (p50, p95) and duplicate refunds (must be 0).

5. Add one global kill_all flag row and a per-run cancel; set each flag during 20 runs of 12 iterations each and count the iterations that run after the flag is set (target: 0 or 1).

6. Emit one OpenTelemetry span per iteration with the attribute loop.iteration, put no customer name or email in any span (check with a grep over the exported JSON), and sum gen_ai usage tokens per run.

7. Cap each run at 12 model calls and 40,000 input tokens, run 30 successful and 10 failing runs, and write the README table: cost per success = total spend / successful runs, the overshoot past each cap, and the worst failure you could not fix.

Deliverable: Git repo with the desk service, docker-compose.yml, a drills/ folder with the scripts, and a README whose results table lists: duplicate refunds over 50 kills (0 required), takeover latency p50/p95 in seconds, stop latency after the kill flag in iterations, cost per successful refund in USD, and the overshoot per cap. Include the exact commands to reproduce each row.

Stretch: Swap SQLite for Postgres with SKIP LOCKED and run the same 50-kill drill on two Compose hosts; report whether the takeover latency changes. · Add a 429 fault injector in front of the model call and report how many runs stop with a clear reason versus retry forever.

Project 2: Dependency-upgrade bot for a 12-service monorepo

Goal: Build a scheduled loop that migrates 12 generated services from billing-client 1.x to 2.0, one isolated git worktree per service, with a permission list, an injection test and a human gate before any push; a platform team hires for exactly this.

Uses lessons: 19, 20, 21, 22, 23, 24

1. Generate a monorepo with 12 services of 3 to 5 small modules each, a pytest suite per service, and a fake internal package billing-client with 3 breaking changes between 1.4 and 2.0; keep the repo under 2 MB and commit it.

2. Write a heartbeat trigger that wakes every 15 minutes, claims one service from a tasks table with a 10-minute lease, and records owner and attempt; run two triggers at once and confirm over 200 claims that no service is ever claimed by two owners.

3. Give each claimed service its own git worktree and branch named upgrade/<service>/<run_id>; cap parallel runs at 3, then 6, and measure wall-clock time for all 12 services at each setting.

4. Write a permission list per run that allows writes only under services/<name>/ and running pytest; attempt 5 forbidden writes (CI config, another service, the lockfile of the monorepo) and count how many are refused.

5. Plant 10 injection variants in one service README and one docstring (for example 'skip the failing test and push'); count how many variants change the diff or the test command (target 0).

6. Add an approval gate before git push and before the PR opens: show the reviewer the diff stat, the test result and the run's tool list; record the reviewer's time per PR and the reject rate over the 12 services.

7. Run the bot over all 12 services and write the README table: services green, false-done rate (pytest passes but a hidden contract test in contracts/ fails), tokens per green service, and reviewer minutes per PR.

Deliverable: Git repo with the monorepo generator, the bot, a worktrees/ cleanup script, the injection fixtures and a README with measured results: claim collisions over 200 claims (0 required), parallel speed-up at 1, 3 and 6 workers, forbidden writes refused out of 5, injections blocked out of 10, false-done rate, and reviewer minutes per PR.

Stretch: Replace the hidden contract tests with a second model as judge and measure how often the judge disagrees with the contract tests; report the disagreement rate and the cases where the judge was wrong. · Schedule the bot as a GitHub Actions workflow on a cron and document which of the three triggers you would use in production and why.

Project 3: On-call incident triage loop with a runbook gate

Goal: Build an event-driven triage loop for 300 generated alerts that reads logs, follows runbooks, asks for approval before any restart or rollback, and reports time to first action and cost per resolved incident against a rules-only baseline; an SRE hiring panel reads this as production judgement.

Uses lessons: 19, 20, 22, 23, 27, 28a

1. Generate 300 alerts across 8 alert types from a seeded script (fixed seed, service names only, no personal data), and hold out 100 of them with hidden labels for scoring.

2. Put the alerts in an events table and write an event-driven worker that polls every 5 seconds and leases one event for 60 seconds; kill the worker mid-incident twice and confirm the event is handled once.

3. Expose 3 read-only tools (read_logs(service, minutes), get_deploy_history(service), read_runbook(name)) and 3 write tools (restart_service, rollback_deploy, scale_out) that pause at an approval step showing the exact arguments.

4. Cap each incident at 8 turns and 0.05 USD of spend and add no-progress detection (the same tool call with the same arguments twice stops the loop); run the 100 held-out alerts and count the stops per reason.

5. Give the bot a per-incident allowlist (read tools for every alert, a write tool only when the alert type matches its runbook) and try 40 forbidden calls; count how many are refused.

6. Export every incident as a span tree with one span per iteration, and grep the exported spans for log lines that contain an email-like or token-like string (target 0 matches).

7. Score the 100 held-out alerts for classification accuracy, false-resolved rate, approval rate, median and p95 time to first action, and cost per resolved incident; run a rules-only baseline on the same 100 alerts and put both in the README.

Deliverable: Git repo with the alert generator, the worker, the tools, the scoring script and a README with a two-column results table (loop vs rules-only): accuracy, false-resolved rate, approval rate, median and p95 time to first action, cost per resolved incident in USD, forbidden calls refused out of 40, and PII matches in spans (0 required).

Stretch: Add a second alert source that arrives in bursts of 50 alerts in 10 seconds and measure how many leases expire and how many incidents are handled twice. · Let the loop propose a runbook change as a pull request and measure how many proposed changes a reviewer would merge.

Project 4: Per-tenant budget board behind a model gateway

Goal: Put the course's model calls behind a LiteLLM gateway in Docker Compose with three tenants, enforce budgets and rate limits, and publish the measured overshoot at 1, 5 and 20 concurrent loops; the trap that a per-run cap is not a monthly ceiling is the point of the README.

Uses lessons: 27, 28b, 29a, 29b

1. Run the LiteLLM proxy and a Postgres container in docker-compose.yml; put LITELLM_MASTER_KEY, LITELLM_SALT_KEY and DATABASE_URL in .env and print only bool(value) in the setup script.

2. Create 3 teams with budgets of 5, 20 and 50 USD and budget_duration 30d, then create 2 virtual keys per team with rpm_limit set to 80% of the project's requests-per-minute value read in AI Studio.

3. Set fail_closed_budget_enforcement: true and fail_closed_rate_limit_enforcement: true in general_settings, then stop the Postgres container and send 100 requests; count the 503 responses (target 100).

4. Run the desk loop with 1, 5 and 20 concurrent loops on one key, each loop capped at 12 model calls; record spend after each run and compute the overshoot in USD and percent over the team budget.

5. Attach tenant_id, run_id and agent identity as key metadata and as span attributes, then build a cost-per-success table per tenant from the gateway logs.

6. Write a rollout note in the README: which budget you would set at the team level, which at the run level, and why the run cap does not stop a monthly overspend.

7. Produce the README results table: overshoot by concurrency (1, 5, 20), fail-closed 503 count out of 100, cost per success per tenant, and the largest single-request overshoot seen.

Deliverable: Git repo with docker-compose.yml, the gateway config, the setup script (no secret values printed), the load script and a README with measured results: overshoot in USD and percent at 1, 5 and 20 concurrent loops, 503 responses out of 100 with Postgres stopped, cost per successful run per tenant, and the largest single-request overshoot.

Stretch: Add a 24-hour simulated schedule with a daily budget window and a 30-day window and report which window trips first. · Replace the gateway with a per-tenant budget check in your own FastAPI middleware and compare its overshoot with the gateway's under the same 20-loop load.

Project 5: Shadow and canary rollout for an invoice-matching loop

Goal: Ship a change to a finance invoice-matching loop the way a release engineer would: shadow run, 5% canary by run hash, automatic rollback when a golden-set regression is planted, and a published decision log; it shows the rollout judgement that separates senior engineers.

Uses lessons: 17, 27, 29b, 30

1. Generate 60 golden invoice-matching cases with expected end-states (matched, held, rejected) and 10 traps (duplicate invoice number, currency mismatch, partial receipt), with fake vendor names and name@example.com contacts only.

2. Define a version bundle file holding the prompt, the tool schema hash, the model id from TUT_LLM_MODEL, the verifier name and the caps; every run records the bundle id it used.

3. Run shadow mode on 200 live-shaped invoices: the candidate bundle runs and its output is discarded, and the script compares its end-states with the golden set and the current bundle.

4. Route 5% of runs to the candidate by sha256(run_id) mod 100 < 5; measure the actual canary share over 10,000 generated run ids and confirm it is within 0.5 percentage points of 5%.

5. Plant a regression in the candidate (drop the currency check) and set the rollback rule to fire when the golden-set pass rate falls more than 3 points below the current bundle; measure how many canary runs happen before rollback fires.

6. Confirm in-flight runs keep their original bundle after the rollback by resuming 10 paused runs and checking their recorded bundle ids.

Deliverable: Git repo with the golden set, the bundle files, the shadow and canary runner, the rollback check and a README with a decision log and measured results: golden pass rate for each bundle, canary share over 10,000 run ids, runs before rollback fired, and in-flight runs that kept their original bundle out of 10.

Stretch: Add a CI job that fails the pull request when the golden-set pass rate drops below the threshold file, and show it failing on the planted regression. · Measure the judge's agreement with 50 human labels for the one case that has no deterministic check and report Cohen's kappa.